Computational Learning Theory

Mihir Rao

Contents

1 Language Identification in the Limit I **2**
- 1.1 Learning from positive examples . . . 2
- 1.2 Computable hypotheses and convergence . . . 2
- 1.3 Examples and extensions . . . 3
- 1.4 Locking sequences . . . 3

Lecture 1: Language Identification in the Limit I

1.1 Learning from positive examples

A teacher has a target language L and presents examples belonging to it. After each example, a learner guesses the entire language. The teacher does not confirm the guess or supply counterexamples outside L. A learner succeeds if its guesses eventually settle on the correct language, even though it need not know when this has happened. This models a form of learning from positive data: observing a string rules out languages that omit it, but does not rule out languages containing additional, as yet unseen strings.

Fix a finite alphabet Σ. A language is a subset of Σ^*, the set of all finite strings over Σ, including the empty string. A positive presentation of L is an infinite sequence $s = (s_1, s_2, \ldots)$ whose range is exactly L: each s_i belongs to L and every member of L appears at least once. Repetitions and arbitrary ordering are allowed. The empty language has no positive presentation under this definition, while every nonempty language has one. Fairness requires eventual appearance, but gives no bound on how long an example may be delayed.

1.2 Computable hypotheses and convergence

Let $\mathcal{L}$ be a collection of languages. A computable indexing $I = (L_1, L_2, \ldots)$ lists exactly the languages in $\mathcal{L}$, possibly with repetitions, and has a single decider that halts on every pair (i, x) and returns whether $x \in L_i$. Thus membership must be decidable uniformly in the index. The learner outputs indices in this fixed list, not the languages as infinite sets.

For a Turing machine M and a presentation s, define

$$m_n = M(\langle s_1, \ldots, s_n \rangle),$$

with $m_n = \perp$ if the computation does not halt. We write $M(s) \to k$ if every m_n is defined and there is a finite N such that $m_n = k$ for all $n > N$. Convergence requires an eventually fixed *index*, not merely a sequence of different indices naming the same language.

Definition 1.1 [Identification in the limit]**.** M identifies $L \in \mathcal{L}$ with respect to I if, for every positive presentation s of L, there is an index k with $L_k = L$ and $M(s) \to k$. The collection $\mathcal{L}$ is identifiable in the limit if there exist one computable indexing and one machine that identify every language in the collection. The final index may depend on the presentation. Because the empty language has no positive presentation, identification of it is vacuous.

1.3 Examples and extensions

All finite languages are identifiable. Use a computable indexing with a computable canonical index for each finite set, and output the index of the observed set $\{s_1, \ldots, s_n\}$. For a nonempty finite target, all its strings appear by some finite time, after which the index stays correct. This learner cannot identify an infinite target: no finite prefix contains it.

Positive presentations can also encode labeled data. For a function $f : X \to Y$ on countable sets, represent its graph as the language $\{\langle x, f(x)\rangle : x \in X\}$. When $Y = \{0, 1\}$, examples with either label appear as positive members of this graph. A sequence is similarly a function on $\mathbb{N}$; its indexed values need not be presented in order.

1.4 Locking sequences

A finite sequence σ of strings from L is a LOCKING SEQUENCE for M on L, locking it to k, if for every finite sequence τ of strings from L, including the empty sequence,

$$M(\langle \sigma \circ \tau \rangle) = k.$$

Once a locking sequence has appeared, no further positive evidence from L can change the output. This is stronger than stability along just one chosen presentation.

Lemma 1.2 [LOCKING SEQUENCE LEMMA]. *Suppose L is nonempty and M identifies L in the limit with respect to a computable indexing (L_i). Then some finite sequence from L locks M onto an index k with $L_k = L$.*

Proof. First, any locking sequence for L must lock onto a correct index: append a positive presentation of L to it. The result is a positive presentation on which M remains at the locked index, so identification forces that index to name L.
Suppose no locking sequence exists. Every finite sequence σ from L can be extended by a finite sequence from L that changes the output. Indeed, M halts on σ, since it is a prefix of some positive presentation; if no extension changed its output, σ would be a locking sequence.
Fix a positive presentation $p_1, p_2, \ldots$ of L. Construct a new sequence by repeatedly appending p_i and then an extension that changes the current output. All its strings lie in L, and every p_i appears, so it is a positive presentation. Yet M changes its output infinitely often on its prefixes, contradicting convergence. Thus a locking sequence exists, and the first observation makes its index correct. □

Source. Jonathan Shafer, Computational Learning Theory, Spring 2021, Lecture 1 and Unit 1 notes.